Project Report Topic:

INTERNET BANKING SYSTEM

Submitted By:

- **Gokul Krishnan**
- **Prakriti Yadav**
- **Sreeja Pamu**
- **Meghana Nanavala**

1. Problem Statement

Modern banking requires secure, convenient, and continuously available digital services. Traditional banking systems often rely on manual processes and disconnected workflows for customer management, transactions, payments, cards, loans, verification, and administration.

The **Internet Banking System** addresses these challenges by providing an integrated digital platform for customer banking and controlled administrative operations.

The system supports:

- Secure registration, login, and two-factor authentication
- Password and session management
- Customer profile and KYC management
- Account and balance management
- Beneficiary management and fund transfers
- Transaction history and statements
- Biller management and bill payments
- Scheduled payments
- Card and loan services
- Notifications, audit logging, and reporting
- Administrative monitoring and approval workflows

Security controls ensure that customers can access only their authorized banking information, while administrative functionality is restricted through role-based access.

The system combines a modular **Oracle JET frontend** with a **Spring Boot microservices backend**. The frontend provides a responsive, validated, and role-aware user experience, while the backend separates banking functionality into specialized services with clearly defined responsibilities.

2. Project Description

The Internet Banking System is a full-stack digital banking application designed to provide banking services to customers and operational capabilities to administrators.

The system consists of two major architectural layers:

1. **Frontend application**
2. **Backend microservices platform**

The frontend is implemented as a single-page application using **Oracle JET** and follows a modular **Model–View–ViewModel (MVVM)** architecture. Views define the user interface, ViewModels manage screen logic, observables, validation, API interaction, and user actions, while shared utilities provide functionality such as API communication, session management, formatting, internationalization, and file downloads.

The backend is implemented using **Java and Spring Boot** and follows a microservices architecture. Rather than placing every banking capability in a single application, banking responsibilities are

separated into independent services such as Authentication, Customer, Account, Transaction, Beneficiary, Bill Payment, Card, Loan, Notification, Audit, Report, Scheduler, Workflow, and Admin services.

A central **API Gateway** connects the two sides of the application.

The browser does not directly communicate with individual backend microservices or their databases. Frontend requests are sent to the API Gateway, which forwards them to the appropriate backend service.

The overall request flow can therefore be represented as:

User
↓
Oracle JET User Interface
↓
Views + ViewModels + Knockout Observables
↓
Shared Frontend API Utility
↓
API Gateway
↓
Backend Microservices
↓
Service-Owned Data / Oracle Database

For operations involving multiple banking domains, backend services interact through secured internal APIs. Kafka is additionally used for asynchronous operations such as notification and audit-event processing.

This architecture separates presentation responsibilities, business logic, persistence, orchestration, security, and asynchronous processing.

3. Technology Stack

The project combines frontend, backend, database, security, messaging, API, build, and deployment technologies.

Category	**Technology**
Backend Programming Language	Java 17
Backend Framework	Spring Boot 3.5.4
Microservice Support	Spring Cloud 2025.0.0
API Gateway	Spring Cloud Gateway

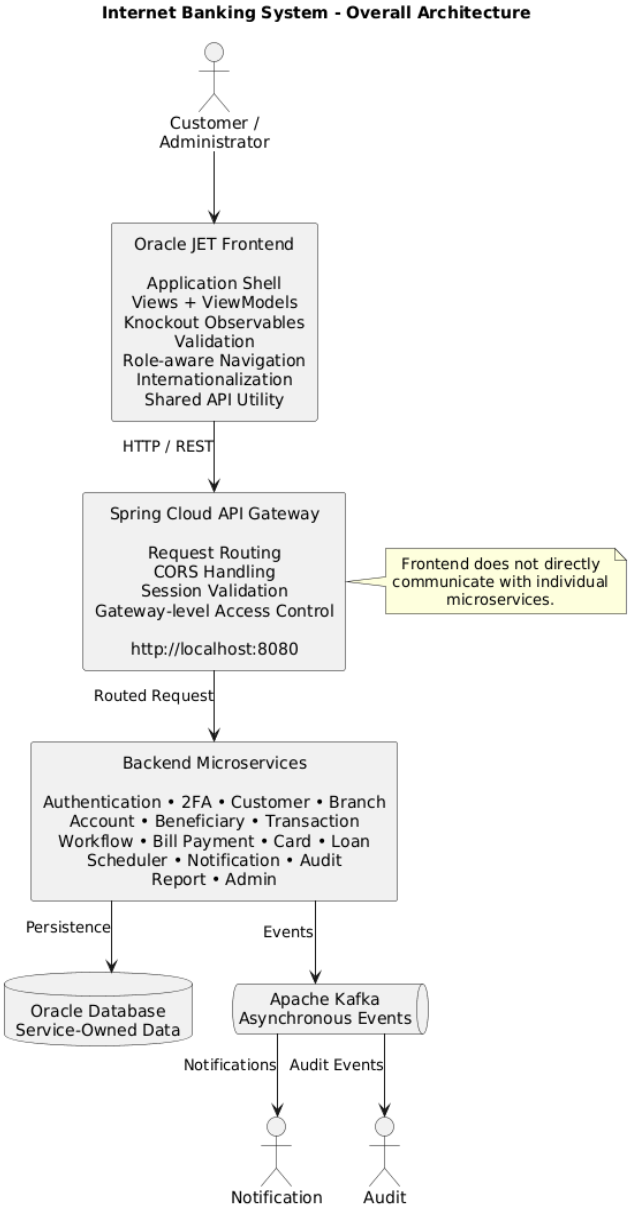
Security Framework	Spring Security
Authentication	JWT
Authorization	Role-Based Access Control
Password Protection	BCrypt
Database	Oracle Database
ORM	Spring Data JPA / Hibernate
Messaging	Apache Kafka
Sensitive Data Encryption	AES-GCM
API Documentation	Swagger / OpenAPI
Backend Build Tool	Maven
Frontend Framework	Oracle JET 20.1.2
Frontend Languages	JavaScript
UI Architecture	MVVM
Data Binding	Knockout.js
UI Components	Oracle JET Components
Styling	CSS with Oracle Redwood Theme
Routing	URL hash-based routing
HTTP Communication	Browser Fetch API
Browser Storage	sessionStorage and localStorage
Internationalization	Oracle JET locale support and JavaScript Intl APIs

Frontend Development Tool	Oracle JET CLI
Node.js Requirement	Node.js 24.19.0
Container Support	Podman Compose

4. Overall System Architecture

The Internet Banking System uses a layered full-stack architecture in which responsibilities are distributed across the browser application, API Gateway, backend domain services, workflow services, asynchronous infrastructure, and persistence layer.

A simplified architecture is:



5. Frontend Architecture

The frontend follows the **Model–View–ViewModel (MVVM)** pattern.

Each banking screen generally consists of:

Component	Responsibility
View	Defines HTML and Oracle JET UI displayed to users
ViewModel	Manages observables, validation, API calls, user actions and UI state
Shared API Utility	Centralizes HTTP requests, JWT attachment, errors, uploads and downloads
Session Utility	Authentication state and session restoration
Application Shell	Navigation, role-based menus, language selection and logout
Oracle JET Components	Responsive and reusable UI components
Knockout.js	Observable-based data binding between View and ViewModel

6. Frontend Structure

The frontend of the Internet Banking System is developed using Oracle JET, Knockout.js, RequireJS, HTML, CSS, JavaScript, and TypeScript. It follows the Model–View–ViewModel (MVVM) architecture, where each banking feature is organized into independent Views and ViewModels.

Application and Module Structure

The main entry point is src/index.html, while src/js/main.js initializes the application and configures RequireJS dependencies. Application-level functionality is managed through root.js and app.js.

The basic application flow is:

index.html → main.js → app.js → Route → View + ViewModel

Each banking module contains:

Component	Responsibility
View	Defines the HTML and Oracle JET user interface
ViewModel	Handles Knockout observables, validation, API calls, UI state, and user actions
Application Module	Manages authentication, sessions, routing, language, and role-based navigation
Shared Utilities	Provides common API, localization, file-download, and UI functionality

Major modules include Login, Registration, Dashboard, Accounts, Transactions, Beneficiaries, Bill Payments, Scheduled Payments, Cards, Loans, Reports, Audit, and Administration.

Oracle JET Components

The frontend uses reusable Oracle JET components to provide a consistent and interactive banking interface. Important component categories include:

- oj-module for dynamically loading Views and ViewModels.
- Input and form components for registration, login, KYC, payments, and other banking forms.
- Selection components, such as oj-select-single, for selecting accounts, billers, and other options.
- Validation components and groups for checking user input before API requests are submitted.
- Message components for displaying success, validation, and error feedback.
- Navigation components for customer and administrator menus.
- Data-display components for accounts, transactions, reports, and dashboard information.

Knockout observables connect these UI components with the corresponding ViewModels so that changes in application data are reflected dynamically in the interface.

Shared Utilities and Routing

Reusable functionality is maintained in src/js/utils/. Important utilities include api.js for:

Component / Utility	**Purpose**
api.js	Handles backend API communication, JWT attachment, error handling, file uploads, and downloads
i18n.js	Supports localization, language selection, and locale-based formatting
File / Statement Utilities	Handles protected file and account-statement downloads
Hash-Based Routing	Dynamically loads frontend modules based on the selected URL route
Customer Routes	#/dashboard, #/accounts, #/transactions, #/bill-payments, #/scheduled-payments
Admin / Supporting Routes	#/reports, #/audit, #/admin
Role-Based Navigation	Controls available menus, routes, and actions according to Customer or Administrator role

7. Backend Architecture

The backend uses a **Spring Boot microservices architecture**.

Layer	**Responsibility**
Controller	Exposes REST endpoints and accepts requests
DTO	Defines API request/response structures

Layer	Responsibility
Service	Implements business rules and domain processing
Repository	Provides persistence through Spring Data JPA
Entity	Maps Java objects to database structures
Exception Handler	Converts failures into structured API errors
Shared Kernel	Reusable constants, validation and common API structures

7. API Gateway and Frontend–Backend Integration

The API Gateway is the central entry point into the backend.

The frontend does not directly call the Authentication Service, Account Service, Transaction Service, Card Service, or other internal services.

Instead:

Frontend → API Gateway → Appropriate Microservice

Service	Port	Main Implementation	Request / Processing Flow	Significance
API Gateway	8080	Routing, CORS, session validation, gateway security	Frontend → Gateway → Target Microservice	Single controlled entry point; hides internal services
Authentication Service	Not specified	Registration, login, password management, JWT, roles, sessions	Credentials → Authentication → JWT → Protected APIs	Manages user identity and authentication
2FA Service	Not specified	OTP, TOTP, QR generation, encrypted authentication factors	Login → 2FA verification → Access	Provides additional authentication security
Customer Service	Not specified	Customer profile and KYC management	Gateway → Customer Service → Profile/KYC	Owns customer and KYC information
Branch Service	Not specified	Branch details and lookup	Request → Branch Service → Branch data	Centralized branch reference information
Account Service	Not specified	Account creation, account status, balances, debit/credit	Workflow → Account validation/operation → Account state	Authoritative owner of account and balance state

Service	Port	Main Implementation	Request / Processing Flow	Significance
Beneficiary Service	Not specified	Beneficiary creation, update, deletion, activation and ownership validation	Customer → Beneficiary Service → Validation	Protects transfer recipient management
Transaction Service	Not specified	Transaction records, status, history and statements	Banking operation → Transaction Service → Transaction record	Source of truth for transaction history
Banking Workflow Service	Not specified	Account opening, deposits, withdrawals, transfers and bill payments	Request → Workflow → Multiple Services → Result	Orchestrates multi-service banking operations
Bill Payment Service	Not specified	Biller catalog, registered billers and payment records	Biller selection → Workflow → Account debit → Payment record	Separates bill-payment logic from core banking
Scheduler Service	Not specified	Scheduled payments, execution and retries	Schedule due → Workflow → Payment execution	Separates when an operation runs from how it runs
Card Service	Not specified	Card applications, issuance, status, limits, activation/blocking	Customer → Card Service → Card processing	Manages card-specific functionality and sensitive card data
Loan Service	Not specified	Applications, loans, EMI schedules and repayments	Application → Loan Service → Loan processing	Isolates loan-specific business rules
Notification Service	Not specified	Notification templates, records and delivery	Event → Kafka → Notification Service → Delivery	Performs notifications asynchronously
Audit Service	Not specified	Audit events and sanitized audit records	Event → Kafka → Audit Service → Audit record	Provides traceability and security monitoring

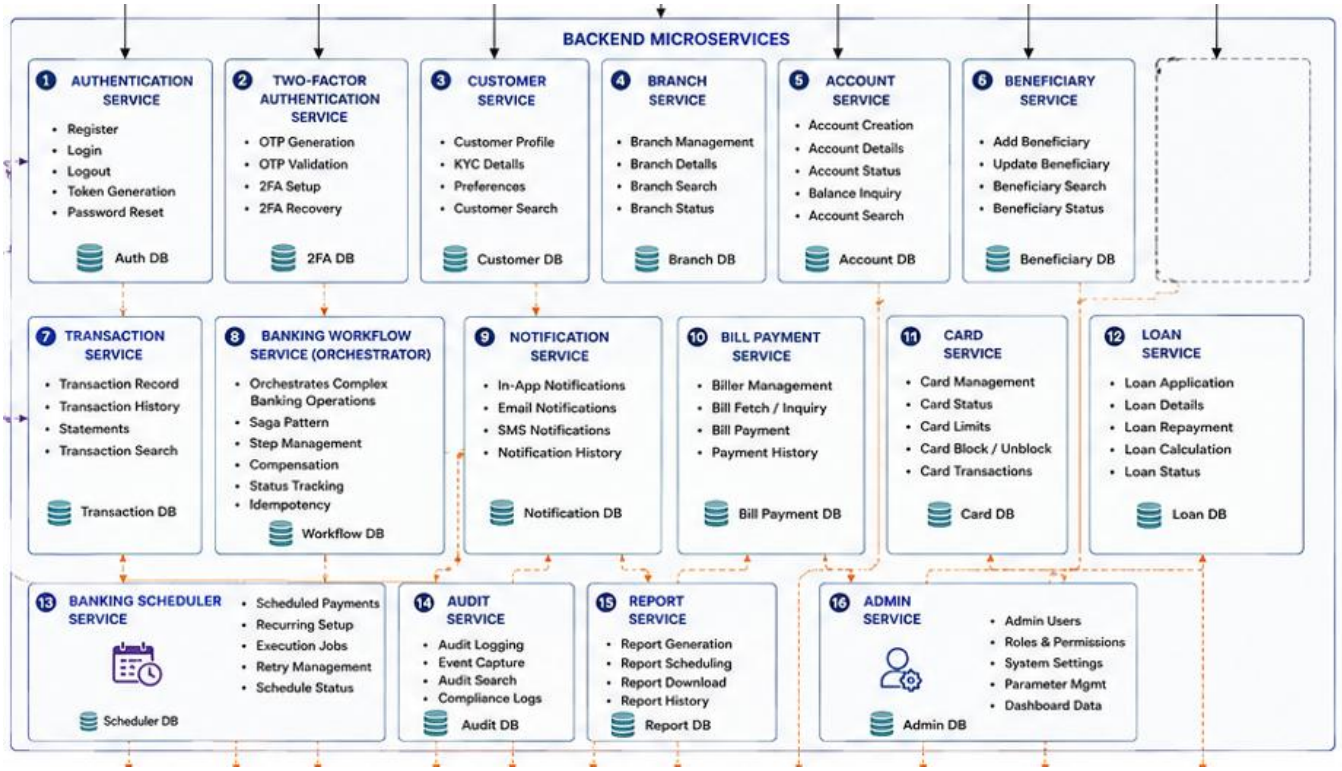


Fig 1. Backend Structure

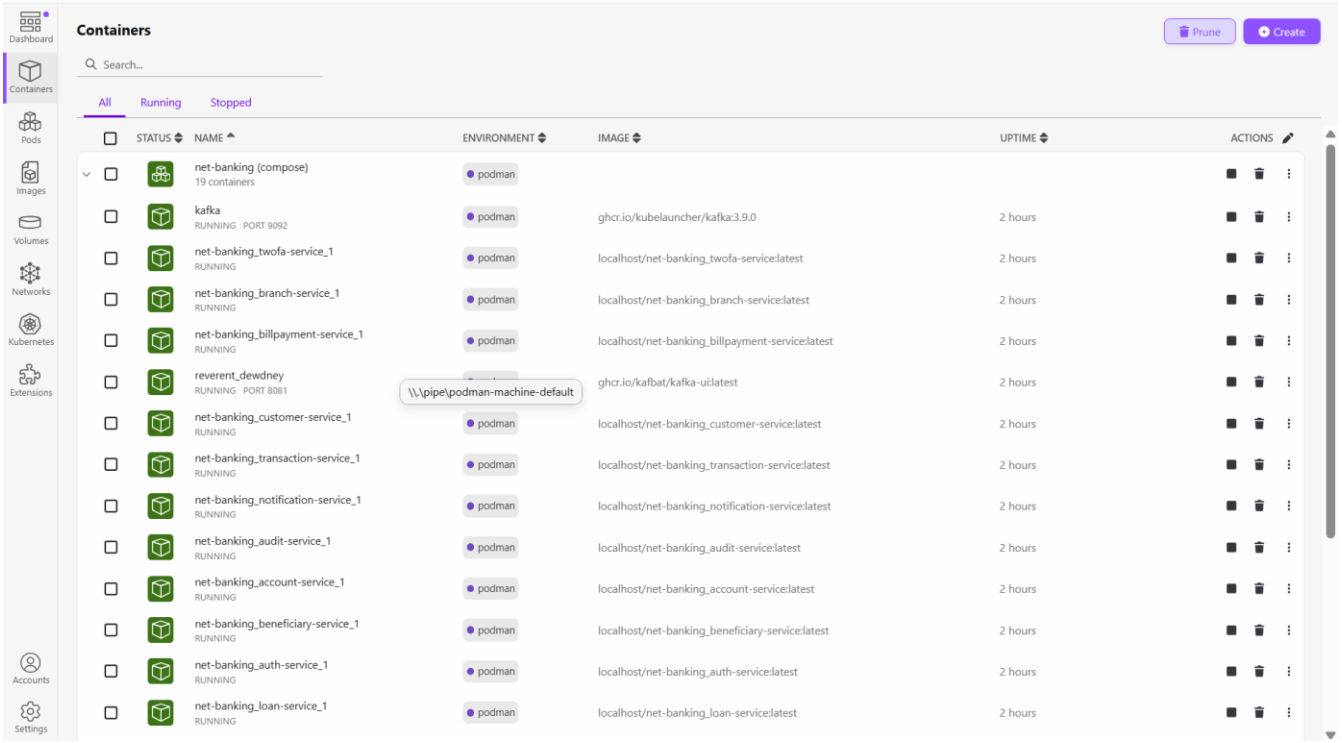


Fig 2. Backend services running as containers

8. AI Usage

Artificial Intelligence tools were used as development-support tools during the project. AI assistance was limited to selected activities and was not used to replace the core implementation or technical decision-making performed by the project team.

Area	AI Usage
Dummy Data Generation	AI was used to assist in generating representative, non-production sample data for development, UI testing, and functional validation.
Documentation	AI was used to assist with structuring, refining, summarizing, and formatting technical documentation, including architectural descriptions and project explanations.
Testing Assistance and Devugging	AI was used to assist in preparing test scenarios, edge cases, sample test data, and validation cases for frontend and backend functionality.

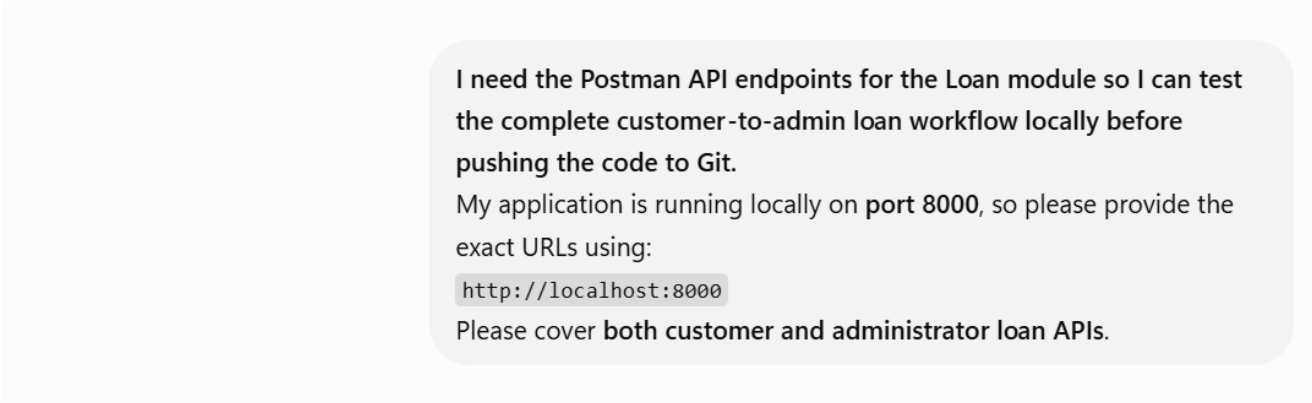


Fig 3. Prompt for testing

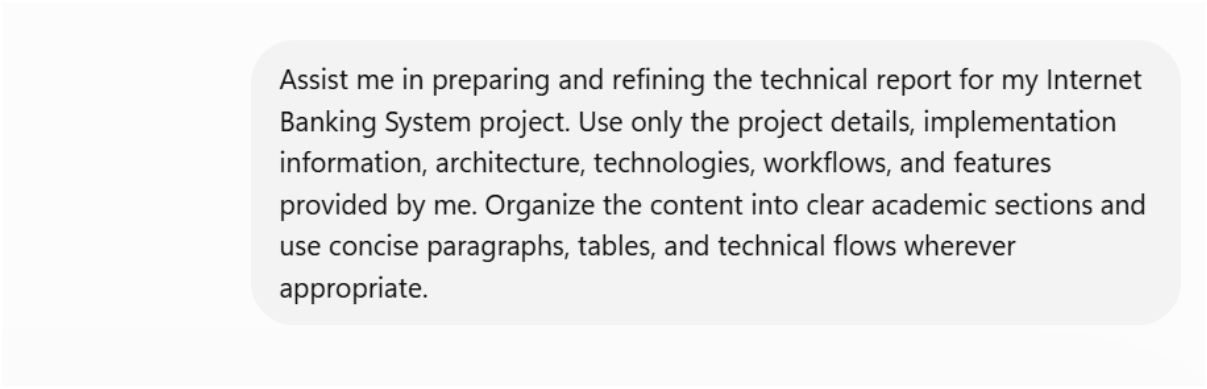


Fig 4. Prompt for report making

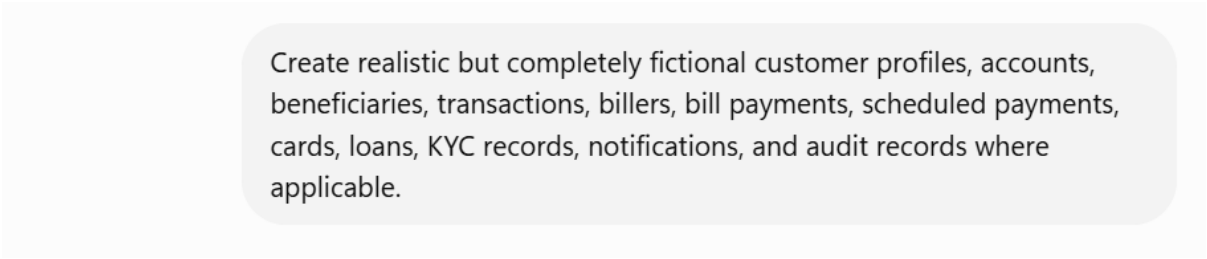


Fig 5. Prompt for Dummy data generation

9. Administrator Features

Area	Features / Responsibilities	Implementation / Significance
Access Control	Role-aware navigation and administrator authorization	Separates administrator functionality from normal customer banking functionality
Admin Dashboard	Customers, accounts, beneficiaries, transactions, banking workflows, loans, cards, branches, bill payments, scheduled payments, audit information, system health	Provides administrators with a consolidated operational view
KYC Approval	Pending KYC reviews; KYC verification	Supports controlled customer verification through backend-protected operations
Beneficiary Management	Pending beneficiary verification; beneficiary approval/blocking	Allows administrators to control beneficiary-related verification and access
Card Management	Pending card applications; card-application approval	Supports administrative review and approval of card applications
Loan Management	Pending loan applications; loan-application approval	Supports administrative review of loan applications
Audit Monitoring	Access to audit information and audit records	Helps administrators monitor user and system activities
System Monitoring	System-health information	Provides operational visibility into the banking platform
Admin Service	Aggregates information from Customer, Account, Transaction, Card, Loan, Audit and other services	Does not own every domain's data; retrieves information through secured internal APIs

Area	Features / Responsibilities	Implementation / Significance
Service Ownership	No direct access to other services' domain tables	Preserves microservice data ownership while providing a unified administrative interface

10. Project Highlights and MVP Implementation

The Internet Banking System includes multiple customer and administrative banking modules. The Minimum Viable Product (MVP) focuses on three key end-to-end workflows: Biller Management and Bill Payments, Scheduled Payments, and KYC Document Verification. These features demonstrate the integration of the Oracle JET frontend, API Gateway, backend microservices, workflow processing, persistence, and administrative operations.

MVP Feature	Implementation	End-to-End Flow	Significance
Biller Management & Bill Payments	Manage billers, select source account, initiate payments, and view payment history.	Customer → Biller → Gateway → Workflow → Account Debit → Payment Record	Demonstrates coordinated multi-service financial processing.
Scheduled Payments	Create, modify, cancel, and execute future or recurring payments.	Customer → Scheduler → Due Execution → Workflow → Payment	Separates payment scheduling from financial execution.
KYC Document Verification	Customer uploads KYC documents; authorized admin views the document and approves or rejects verification.	Customer → KYC Upload → Customer Service → Admin Review → Approve/Reject → Status Update	Demonstrates an end-to-end customer-to-admin verification workflow with role-based access.

11. Functional Requirements

The combined functional requirements include:

Module	Functional Requirements
Authentication	Registration, login, 2FA, password reset, sessions, role-based access
Customer	Profile, personal information, KYC
Accounts	Creation, viewing, status, balances, controlled debit/credit
Beneficiaries	Add, update, delete, activate/deactivate, validation
Transactions	Transfers, deposits/withdrawals, history, search, statements
Bill Payments	Biller catalog, registration, management, payment and history

Module	Functional Requirements
Scheduled Payments	Future/recurring payments, modification, cancellation, execution history
Cards	Application, viewing, masking, status, activation and blocking
Loans	Application, status, EMI, repayment history and outstanding balance
Administration	Dashboard, approvals, audits, reports and system information
Supporting Services	Notifications, audit logging, reporting and branch lookup

12. Future Scope

- **Fraud Detection** – Analyze transaction patterns and flag suspicious transactions.
- **Real-Time Notifications** – Add SMS, push notifications, WebSockets, or Server-Sent Events.
- **External Payments** – Integrate UPI and external payment gateways.
- **Credit-Score Integration** – Improve loan eligibility and assessment.
- **CI/CD Pipeline** – Automate application build, testing, and deployment.
- **Centralized Monitoring** – Add centralized logs, metrics, dashboards, and alerts.

13. Conclusion

The **Internet Banking System** demonstrates the design and implementation of a secure, modular, and integrated full-stack digital banking platform. The system brings together customer banking operations, administrative capabilities, security mechanisms, and supporting services within a clearly separated architecture.

The **Oracle JET frontend**, based on the MVVM architecture, provides structured interfaces for both customers and administrators. It supports essential banking functions including registration and authentication, two-factor verification, account management, beneficiaries, fund transfers, transaction history, bill and scheduled payments, cards, loans, reports, audit exploration, and administrative workflows. Responsive design, validation, localization, session management, and consistent error handling further improve the overall user experience.

The **Spring Boot backend** follows a microservices architecture in which major banking domains are implemented as dedicated services. The **API Gateway** acts as the controlled integration point between the frontend and backend, while individual microservices remain responsible for their respective business rules and service-owned data. Complex financial operations, particularly fund transfers and bill payments, are coordinated through the **Banking Workflow Service**, ensuring that responsibilities remain clearly separated across banking domains.

Security is incorporated throughout the system using **JWT authentication, BCrypt password hashing, role-based access control, two-factor authentication, sensitive-data encryption, session validation, masked card information, secured internal communication, idempotency controls, and audit logging**. In addition, **Apache Kafka** supports asynchronous notification and audit-event

processing, while asynchronous report generation prevents long-running operations from unnecessarily blocking normal banking requests.

Although the current system is an academic prototype rather than a production banking platform, its modular architecture provides a strong foundation for future development. Potential enhancements include fraud detection, UPI and external payment integration, real-time notifications, credit-score integration, enhanced card-security workflows, automated testing, CI/CD, centralized monitoring, distributed tracing, and secure cloud deployment.

Overall, the project demonstrates how a modern digital banking application can combine a structured user experience with secure and maintainable backend services. Its core architectural principle is **separation of responsibilities**: the frontend manages the user experience, the API Gateway controls the integration boundary, domain microservices manage banking rules and data, the Workflow Service coordinates complex financial operations, and asynchronous infrastructure supports activities such as notifications and auditing.